

REMOTE_DEMO_PLAYBOOK

NetworkPeer Remote Demo Playbook

Complete guide for demoing the NetworkPeer Android app on a work Linux laptop tomorrow.

TL;DR — Quick Start (5 minutes)

```
# 1. Transfer the APK to your work laptop (USB, Slack, Google Drive, or SCP)
# APK location on your Mac:
  ~/Desktop/NetworkPeer_Demo/NetworkPeer-v0.1.0-dev.apk

# 2. On the work Linux laptop:
# Option A: If you have a physical Android phone
adb install -r NetworkPeer-v0.1.0-dev.apk

# Option B: If you have Android Studio with an AVD
# Open Android Studio → Devices → Run the APK

# Option C: Web demo (no Android needed)
# Open browser → https://network-peer-web.vercel.app
```

1. What You Need

Item	Where to Get It	Priority
APK file	~/Desktop/NetworkPeer_Demo/NetworkPeer-v0.1.0-dev.apk (33MB)	Required

Item	Where to Get It	Priority
Android phone OR Linux emulator	Physical phone with USB cable, or Android Studio	Required
adb on Linux	<code>sudo apt install android-tools-adb</code>	Required
Web browser	Chrome/Firefox for web fallback demo	Backup

How to Transfer the APK to Your Work Laptop

Option 1: USB (fastest)

```
# On your Mac, connect USB cable
# The APK is at:
~/Desktop/NetworkPeer_Demo/NetworkPeer-v0.1.0-dev.apk
```

Option 2: SCP/SFTP

```
# From work laptop (if Mac is accessible on same network):
scp adityasharma@<mac-ip>:~/Desktop/NetworkPeer_Demo/NetworkPeer-
v0.1.0-dev.apk .
```

Option 3: Slack/Email/Drive - Upload `NetworkPeer-v0.1.0-dev.apk` to Slack DM, Google Drive, or email it to yourself - Download on work laptop

Option 4: GitHub Release (if repo is accessible)

```
# On work laptop:
git clone https://github.com/addy9087/Networkpeer.git
# The APK is NOT in git, but you can build it:
cd apps/android
sudo apt install openjdk-17-jdk
./gradlew assembleDevelopmentDebug
# APK will be at: app/build/outputs/apk/development/debug/app-
development-debug.apk
```

2. Demo Setup on Linux (3 Options)

Option A: Physical Android Phone + scrcpy (RECOMMENDED)

This mirrors your phone screen to the Linux laptop at 60 FPS — perfect for presentations.

```
# Install scrcpy
sudo apt install scrcpy

# Enable USB Debugging on your phone:
# Settings → About Phone → Tap "Build Number" 7 times
# Settings → Developer Options → Enable "USB Debugging"

# Connect phone via USB, then:
adb devices          # Should show your device
adb install -r NetworkPeer-v0.1.0-dev.apk
scrcpy --stay-awake --turn-screen-on --power-off-on-close=false
```

Demo flow on phone: 1. Open NetworkPeer app 2. Show the landing page / role selection 3. Walk through "Post a Job" wizard (6 steps) 4. Show the review queue 5. Demonstrate approve/redo/reject workflow

Option B: Android Emulator on Linux (KVM)

```
# Check KVM support (must show "kvm" in output)
egrep -c '(vmx|svm)' /proc/cpuinfo

# Install Android Studio
sudo snap install android-studio --classic

# Create AVD:
# Android Studio → More Actions → Virtual Device Manager → Create Device
# Select: Pixel 7 → API 36 (Android 16) → arm64-v8a or x86_64 image

# Launch emulator from command line:
emulator -avd <avd_name> -gpu host -no-audio

# Install APK:
adb install -r NetworkPeer-v0.1.0-dev.apk
adb shell monkey -p com.networkpeer.mobile.dev -c android.intent.category.LAUNCHER 1
```

Option C: Waydroid (Container-based, no emulator overhead)

```
# Install Waydroid (Ubuntu 22.04+)
sudo apt install curl ca-certificates
curl -s https://repo.waydro.id | sudo bash
sudo apt install waydroid

# Initialize with GAPPS
sudo waydroid init -s GAPPS -f

# Start Waydroid
sudo systemctl start waydroid-container
waydroid session start

# Install APK inside Waydroid
waydroid app install NetworkPeer-v0.1.0-dev.apk
```

Option D: Web Demo Fallback (No Android Needed)

If you can't set up Android on the work laptop, demo the web version:

```
# Open in browser
# https://network-peer-web.vercel.app

# Demo flow:
# 1. Landing page → Show platform overview
# 2. Post a Job → Walk through the 6-step wizard
#   - Job Info (title, description, location)
#   - Task Type (collection/correction)
#   - Media Requirements (photo/video specs)
#   - Capacity Mode (single/capped/unlimited)
#   - Escrow Setup (per-unit pricing)
#   - Review & Submit
# 3. Review Queue → Show submission review pane
#   - Zoom/pan image viewer
#   - OCR results panel
#   - Approve/Redo/Reject actions
# 4. Worker Dashboard → Show assignments
```

3. Demo Script (What to Show Your Boss)

Act 1: Platform Overview (2 minutes)

"NetworkPeer is a peer-to-peer job platform where clients post collection jobs and workers accept and complete them with evidence review."

- Show the landing page with three portals: Client, Worker, Admin
- Mention the tech stack: React/TypeScript frontend, Kotlin/Swift mobile, Fastify API, PostgreSQL + PostGIS, Redis, AWS ECS

Act 2: Post a Job (3 minutes)

"Let me walk through how a client posts a job."

Navigate through the 6-step wizard: 1. **Job Info**: Enter title "Street Sign Collection - Downtown", description, location 2. **Task Type**: Select "Collection" (worker photographs real-world objects) 3. **Media Requirements**: Show photo specs (min resolution, edge coverage, etc.) 4. **Capacity Mode**: Explain: - **Single**: One worker only - **Capped**: Up to N

workers (e.g., 5) - **Unlimited**: Any number of workers 5. **Escrow Setup**: Show per-unit pricing (e.g., \$2.50 per sign photographed) 6. **Review & Submit**: Preview the job card

Act 3: Worker Workflow (3 minutes)

"Now let's see how a worker picks up and completes the job."

1. Show the review queue with available jobs
2. Accept a job → Show the assignment card
3. Open the in-app camera → Show the capture interface
4. Submit a photo → Show the quality check (edge detection, blur, exposure)
5. Show the submission status (pending review)

Act 4: Review & Approval (3 minutes)

"The client reviews submissions with our split-pane review tool."

1. Open the SubmissionReviewPane
2. Show the left panel: zoomable image viewer with pan/zoom
3. Show the right panel: OCR results, quality metrics, confidence scores
4. Demonstrate the three actions:
 - **Approve**: Submission accepted, payment released
 - **Redo**: Worker asked to retake (with note)
 - **Reject**: Dispute, escalation to admin
5. Show the thumbnail strip for quick navigation

Act 5: Architecture & Security (2 minutes)

"Let me show you the architecture."

- Show the GitHub repo structure
 - Point out the contracts-first approach (packages/contracts)
 - Mention the CI/CD pipeline (GitHub Actions → ECS)
 - Highlight security: Cognito auth, no PII before acceptance, escrow safety
-

4. Testing on Your Mac M1 (Android + iOS)

Test the Android App on Mac M1

Prerequisites installed: - Java 17 (OpenJDK via Homebrew) - Android SDK (platforms;android-36, build-tools;35.0.0, platform-tools)

Build the APK:

```
export JAVA_HOME="/opt/homebrew/Cellar/openjdk@17/17.0.20.1/libexec/openjdk.jdk/Contents/Home"
export ANDROID_HOME="$HOME/Library/Android/sdk"

cd apps/android
./gradlew assembleDevelopmentDebug --no-daemon

# APK output:
# app/build/outputs/apk/development/debug/app-development-
#   debug.apk (33MB)
```

Test on a Physical Android Phone:

```
# 1. Enable USB Debugging on your phone:
#   Settings → About Phone → Tap "Build Number" 7 times
#   Settings → Developer Options → Enable "USB Debugging"

# 2. Connect phone via USB cable

# 3. Verify connection:
adb devices
# Should show: XXXXXXXX    device

# 4. Install the APK:
adb install -r
    apps/android/app/build/outputs/apk/development/debug/app-
    development-debug.apk

# 5. Launch the app:
adb shell monkey -p com.networkpeer.mobile.dev -c
    android.intent.category.LAUNCHER 1

# 6. (Optional) Mirror phone to Mac screen:
brew install scrcpy
scrcpy --stay-awake --turn-screen-on
```

Test on Android Emulator (no physical phone needed):

```
# 1. Install Android Studio:
brew install --cask android-studio

# 2. Open Android Studio → More Actions → Virtual Device Manager
# 3. Create device: Pixel 7 → API 36 (arm64-v8a) → Finish
# 4. Start the emulator

# 5. Install APK:
```

```
adb install -r
  apps/android/app/build/outputs/apk/development/debug/app-
  development-debug.apk

# 6. Launch:
adb shell monkey -p com.networkpeer.mobile.dev -c
  android.intent.category.LAUNCHER 1
```

Test the iOS App on Mac M1

Prerequisites: - Xcode 15+ (install from App Store or `xcode-select --install`) - CocoaPods (`sudo gem install cocoapods`)

Build and run on Mac:

```
cd apps/ios

# Install dependencies (if using CocoaPods)
pod install

# Open in Xcode
open NetworkPeer.xcworkspace

# In Xcode:
# 1. Select "My Mac (Designed for iPad)" as the run destination
# 2. Press ⌘R to build and run
# 3. The app will launch in a resizable window on your Mac
```

Test on iPhone (physical device):

```
# 1. Connect iPhone via USB cable
# 2. Trust the computer on your iPhone
# 3. In Xcode: Window → Devices and Simulators → select your
  iPhone
# 4. Select your iPhone as run destination
# 5. Press ⌘R to build and deploy to iPhone
# 6. On iPhone: Settings → General → VPN & Device Management →
  Trust your developer profile
```

Quick iOS Test (Simulator only):

```
cd apps/ios
xcodebuild -scheme NetworkPeer -destination 'platform=iOS
  Simulator,name=iPhone 15' build
# Then open the .app in Xcode's simulator
```

API Configuration for Local Testing

The Android app points to `https://network-peer-api-alpha.vercel.app/api/v1/` by default. To test against a local backend:

```
# 1. Start the backend locally:
cd NetworkPeer-main
```

```
npm install
npm run dev # Runs on http://localhost:3000

# 2. Update Android config:
# Edit apps/android/networkpeer.development.local.properties:
# API_BASE_URL=http://10.0.2.2:3000/api/v1/
# (10.0.2.2 is the Android emulator's alias for host machine)

# 3. Rebuild and reinstall:
cd apps/android
./gradlew assembleDevelopmentDebug
adb install -r app/build/outputs/apk/development/debug/app-
development-debug.apk
```

5. Troubleshooting

Issue	Fix
<code>adb devices</code> shows nothing	Enable USB Debugging, try different cable, install udev rules
APK install fails	<code>adb uninstall com.networkpeer.mobile.dev</code> then retry
App crashes on launch	Check logcat: <code>adb logcat -s NetworkPeer</code>
Emulator is slow	Use <code>-gpu host</code> flag, allocate 4GB+ RAM
Web demo loads but API fails	API is on AWS ECS, may need VPN; use localhost mock if needed

Linux udev Rules for Android Phone

```
# Create /etc/udev/rules.d/51-android.rules
echo 'SUBSYSTEM=="usb", ATTR{idVendor}=="18d1", MODE="0666",
      GROUP="plugdev"' | sudo tee /etc/udev/rules.d/51-
      android.rules
sudo udevadm control --reload-rules
sudo udevadm trigger
```


5. Key Talking Points

1. **Contracts-first:** All data models defined in TypeScript before implementation
 2. **Privacy-by-default:** No PII collected until worker accepts the job
 3. **Escrow safety:** Funds held in escrow, released only on approved submissions
 4. **Quality pipeline:** Automated checks for edge coverage, blur, exposure
 5. **Multi-role review:** Correctionists fix data, clients approve, admins arbitrate
 6. **Offline-capable:** Mobile apps work offline with sync when connected
 7. **Real-time updates:** WebSocket-based live status across devices
-

6. Files Included in Demo Package

```
~/Desktop/NetworkPeer_Demo/  
├─ NetworkPeer-v0.1.0-dev.apk      # Android debug APK  
  (33MB)  
├─ (your notes/prep files)
```

Repository Links

- **Main repo:** <https://github.com/addy9087/Networkpeer>
- **Partner repo:** <https://github.com/rudraaxl/NetworkPeer>
- **Feature branch:** `feature/platform-overhaul-vipul-sync`
- **Web demo:** <https://network-peer-web.vercel.app>
- **PR URL (rudraaxl):**
<https://github.com/rudraaxl/NetworkPeer/pull/new/feature/platform-overhaul-vipul-sync>
- **PR URL (addy9087):**
<https://github.com/addy9087/Networkpeer/pull/new/feature/platform-overhaul-vipul-sync>